AI Document Scanner

Quick guide: how it works, how to train it, and the API

1. What it does

Upload a document (PDF, image, Word, PowerPoint, text, or a zip of them). The scanner reads it, decides what kind of document it is, and returns JSON: **type, confidence, extracted fields, expiry status and country**. Built-in categories are **Vehicles, Company, Equipments** and **People**. Anything else is grouped into an auto-created category, and new categories can be taught from the **Training page** (`/train`) with no code change and no restart.

2. Architecture

Component	Where	Role
Laravel frontend	frontend/, port 8001	Web pages (scan, search, train), public JSON API, SQLite store of scanned documents, zip handling.
FastAPI parser	backend/, port 8000	Reads and classifies documents. Instant endpoint, queue endpoint, training endpoints. Needs PARSER_API_KEY.
Redis + RQ worker	backend/app/tasks.py	Runs the same scan pipeline in the background. Start more workers to scale.
Data files and models	backend/data/	Built-in and learned categories, MiniLM embedding model, Tesseract OCR.

Flow: **Browser / app -> Laravel (API_KEY) -> FastAPI (PARSER_API_KEY) -> instant answer, or Redis queue -> worker**.

3. Scan flow (one function: parse_document)

- **Validate** the file (supported type, at most 10 MB).
- **Extract text**: pdfplumber for PDFs, direct read for Word / PowerPoint / text.
- **OCR fallback** (Tesseract, English) when native text is under 30 characters. PDFs: 200 DPI, first 5 pages.
- **Classify** (keywords, then embeddings, then filename hints), then **table extraction** (PDF) and a refined second pass.
- No match: place in an **auto-created category**.
- **Extract metadata** by category, merge table values, then detect **expiry** and **country**.
- Return one JSON object. Every category found in the document gets its own entry in `categories[]`.

4. Classification

method	How it decides	Confidence
keyword	Counts category keywords in the text; best category wins with 2 or more matches. One 'pat test' style phrase means Equipments.	matched / total keywords (PAT: 0.55)
embedding	MiniLM embedding compared (cosine) with sample texts of each category.	similarity, minimum 0.55
filename	Known words in the file name (light, extinguisher, pat test).	0.55
auto	Grouped with a similar auto category, or a new one is created from the title line.	similarity, minimum 0.5

5. Training (no code, no restart)

- Open **Train categories**, type a category name (new, or an existing one to improve it), drop 3 to 10 sample documents or a zip, press **Train**.
- The scanner learns **distinctive keywords** (up to 15 per run) and **text samples**; learned data is saved in `backend/data/trained_categories.json`. Built-in files are never modified.
- The API server and every worker reload it automatically, so the next scan already uses it.
- **Confirm** turns an auto-created category into a trained one. **Delete** removes it. **Reset training** on a built-in category removes only the extra training.
- Categories without a dedicated extractor return generic `Label: value` fields as metadata.

6. API

Laravel API (applications call this): base `http://127.0.0.1:8001/api`, header `X-API-Key: <API_KEY>`. Wrong or missing key: 401. Key not configured: 503.

Method	Path	Purpose
POST	/api/scan	Upload one document or a zip (field **file**). 202 with {"batch_id": "..."}.
GET	/api/scan/{batch}/status	{"done": false, "finished": 1, "total": 3}. 404 if expired. Poll every 1-2 s.
GET	/api/scan/{batch}/result	Final results, single use. 409 if not finished.
GET	/api/search?q=&type=&status=	Search all scanned documents (max 50). type=unknown lists unclassified.

```
curl -X POST http://127.0.0.1:8001/api/scan -H "X-API-Key: $KEY" -F "file=@passport.pdf"
curl http://127.0.0.1:8001/api/scan/<batch_id>/status -H "X-API-Key: $KEY"
curl http://127.0.0.1:8001/api/scan/<batch_id>/result -H "X-API-Key: $KEY"
```

Parser API (single document, port 8000): header `X-API-Key: <PARSER_API_KEY>`. Open without key: /health, /docs.

Method	Path	Purpose
POST	/parse-document	One document, result returned in the same request (field **file**).
POST	/jobs	Queue one document. 202 with job_id. Then GET /jobs/{job_id}: queued / started / finished (+result) / failed.
POST	/debug-keywords	Per-category keyword scores, for diagnosing a wrong classification.
GET	/categories	All categories (built-in, trained, auto) with keywords.
POST	/train	Form fields **category** and one or more **files**.
DELETE	/categories/{name}	Delete a trained / auto category, or reset a built-in one's extra training.
POST	/categories/{name}/promote	Confirm an auto category. Optional JSON {"name": "New name"}.

```
curl -X POST http://127.0.0.1:8000/parse-document -H "X-API-Key: $PARSER_KEY" -F "file=@rc_book.pdf"
```

Result object (values illustrative; fields not found are null):

```
{
  "success": true, "document_type": "Vehicles", "confidence": 0.62, "method": "keyword",
  "keywords_detected": ["registration certificate", "engine number"],
  "country": "India", "text_length": 1450, "processing_time_ms": 820,
  "metadata": { "vehicle_number": "DL01AB1234", "owner_name": null },
  "expiry": { "start_date": "2026-04-01", "expiry_date": "2027-04-01",
              "days_remaining": 152, "status": "valid" },
  "categories": [ { "category": "Vehicles", "confidence": 0.62,
                    "keywords_detected": ["engine number"], "metadata": {} } ]
}
```

Errors: {"success": false, "error_code", "message"}. Codes: unsupported_file_type 400, invalid_file 400 (empty or over 10 MB), corrupt_or_unreadable_file 422, ocr_failure 422, invalid_category 400, category_not_found 404, job_not_found 404, queue_unavailable 503 (Redis down), internal_error 500.
Limits: Laravel accepts 20 MB per document, 50 MB per zip, 10 MB per file inside a zip, 100 files per zip. The parser itself rejects files over 10 MB.

7. Run it

```
# Redis must be running:  brew services start redis
# 1. Backend (port 8000)
cd backend && source venv/bin/activate && uvicorn app.main:app --port 8000
# 2. Worker (add more terminals to scale)
cd backend && source venv/bin/activate && rq worker parse --worker-class rq.SimpleWorker
# 3. Frontend (port 8001, allows 50 MB uploads)
cd frontend && ./serve.sh 8001
```

File	Variable	Purpose
backend/.env	PARSER_API_KEY	Secret callers send to the parser.
frontend/.env	PARSER_URL, PARSER_API_KEY	Where the parser is, and its key (same as backend).
frontend/.env	API_KEY	Secret API clients send to the Laravel API. Never put it in browser code.
frontend/.env	APP_PASSWORD, SCAN_MAX_ZIP_FILES	Web page password (empty = open); files per zip (default 100).

After changing backend code, restart the backend and every worker. Prerequisites: Python 3.11, Tesseract, Poppler, Redis, PHP 8.3+, Composer.

8. Known limits

- Field extraction is detailed only for Vehicles, Company, Equipments and People; other categories get generic `Label: value` fields.
- OCR is English only; only the first 5 PDF pages are read; training runs synchronously.
- Learned data is stored in local JSON files: use a persistent volume in production. Earlier scans are not re-classified after training.